tf.compat.v1.nn.weighted_cross_entropy_v Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/weighted_cross_entropy_with_logits

> *Computes a weighted cross entropy. (deprecated arguments)*

Function Signatures

```
tf.compat.v1.nn.weighted_cross_entropy_with_logits(
labels=None, logits=None, pos_weight=None, name=None,
targets=None )
(targets)
sigmoid_cross_entropy_with_logits()
labels * -log(sigmoid(logits)) + (1 - labels) * -log(1 -
sigmoid(logits))
```

Code Examples

```
tf.compat.v1.nn.weighted_cross_entropy_with_logits(
labels=None, logits=None, pos_weight=None, name=None, targets=None
)
```

```
tf.compat.v1.nn.weighted_cross_entropy_with_logits(
labels=None, logits=None, pos_weight=None, name=None, targets=None
)
```

```
labels * -log(sigmoid(logits)) +
(1 - labels) * -log(1 - sigmoid(logits))
```


```
labels * -log(sigmoid(logits)) +
(1 - labels) * -log(1 - sigmoid(logits))
```


```
labels * -log(sigmoid(logits)) * pos_weight +
(1 - labels) * -log(1 - sigmoid(logits))
```


```
labels * -log(sigmoid(logits)) * pos_weight +
(1 - labels) * -log(1 - sigmoid(logits))
```


```
qz * -log(sigmoid(x)) + (1 - z) * -log(1 - sigmoid(x))
= qz * -log(1 / (1 + exp(-x))) + (1 - z) * -log(exp(-x) / (1 +
exp(-x)))
= qz * log(1 + exp(-x)) + (1 - z) * (-log(exp(-x)) + log(1 + exp(-
x)))
= qz * log(1 + exp(-x)) + (1 - z) * (x + log(1 + exp(-x))
= (1 - z) * x + (qz +  1 - z) * log(1 + exp(-x))
= (1 - z) * x + (1 + (q - 1) * z) * log(1 + exp(-x))
```


```
qz * -log(sigmoid(x)) + (1 - z) * -log(1 - sigmoid(x))
= qz * -log(1 / (1 + exp(-x))) + (1 - z) * -log(exp(-x) / (1 +
exp(-x)))
= qz * log(1 + exp(-x)) + (1 - z) * (-log(exp(-x)) + log(1 + exp(-
x)))
= qz * log(1 + exp(-x)) + (1 - z) * (x + log(1 + exp(-x))
= (1 - z) * x + (qz +  1 - z) * log(1 + exp(-x))
= (1 - z) * x + (1 + (q - 1) * z) * log(1 + exp(-x))
```

Documentation

Computes a weighted cross entropy. (deprecated arguments)

This is like sigmoid_cross_entropy_with_logits() except that pos_weight,
allows one to trade off recall and precision by up- or down-weighting the
cost of a positive error relative to a negative error.

The usual cross-entropy cost is defined as:

A value pos_weight > 1 decreases the false negative count, hence increasing the recall.
Conversely setting pos_weight < 1 decreases the false positive count and increases the precision.
This can be seen from the fact that pos_weight is introduced as a
multiplicative coefficient for the positive labels term
in the loss expression:

For brevity, let x = logits, z = labels, q = pos_weight.
The loss is:

Setting l = (1 + (q - 1) * z), to ensure stability and avoid overflow,
the implementation uses

logits and labels must have the same type and shape.

Returns

Raises